Git (No confundir con GitHub)

Citruxs

4 de marzo de 2026

¿Qué es Git?

Git es un software de control de versiones, pensando en la eficiencia, la confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones cuando están tienen un gran número de archivos.

Algunos beneficios de su uso son:

- Desarrollo distribuido
- Historial de cambios
- Rastreo de errores

Instalación de Git

La instalación de Git dependerá del sistema operativo en el que te encuentres

- Windows: `https://git-scm.com/download/win`
- MacOS: Usa Homebrew: brew install git
- Linux: sudo apt-get install git (Ubuntu/Debian), sudo dnf install git (CentOS/RHEL)

Para verificar la versión de git con la que cuentas solo hace falta escribir sobre la terminal 'git –version'

Configuracion de Git

Una vez que Git está instalado en nuestro sistema solo hace falta configurar el nombre y correo electrónico con el que los commits serán asociados.

- git config –global user.name “Robocop Pérez”
- git config –global user.email “robocop.perez@example.com”

Creación y uso de un repositorio

Una manera sencilla de iniciar un repositorio es ubicarnos sobre la carpeta donde estará alojado nuestro repositorio, abrir la terminal y escribir:

- git init

Podemos verificar que el repositorio esta hecho con:

- git status

Una buena practica para el uso de repositorios en Git es agregar un archivo readme, pues en este se suele agregar un resumen sobre el repositorio en el que se trabaja, así como instrucciones en general, el formato de este archivo debe de ser .md

Git tiene tres etiquetas distintas para identificar el estatus en el que se encuentran los archivos dentro del repositorio

- modificado: Son los archivos que hemos modificado desde la ultima versión “oficial”que tenemos en el repositorio.
- preparado (stagged) : Son los archivos a los que ya hemos hecho modificaciones pero con la diferencia de que está preparado para reportar los cambios.
- confirmado (committed): Son los archivos los cuales ya tienen los cambios reportados.

Veamos un ejemplo creando nuestro archivo README.md en un repositorio de prueba

Una vez que hemos creado el archivo README.md o hecho algún cambio sobre el repositorio podemos verificar los cambios

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Así vemos que en la sección de untracked files nos indica que el archivo README.md tiene cambios pero no están siendo reportados en el repositorio.

Reportaremos los cambios del archivo README.md, para esto usamos la siguiente instrucción

- git add README.md
- Una alternativa es usar 'git add .', con esta instrucción le indicamos a git que agregue todos los cambios que hemos hecho en todo el repositorio (No es recomendable usarlo si hemos echo diferentes tipo de cambios)

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git add README.md
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Una vez que los cambios están listos para ser reportados, comentamos brevemente los cambios que hemos hecho, para esto usamos:

- git commit -m “Inserte un comentario ingenioso”

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git commit -m "Se agrega el archivo README"
[master (root-commit) 05af29a] Se agrega el archivo README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git status
On branch master
nothing to commit, working tree clean
```

Y listo, nuestros cambios han sido correctamente reportados y podemos visualizar el historial de cambios con:

- git log

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git log
commit 05af29a76db53c6e3b581bd7cc1da32b72f40d76 (HEAD -> master)
Author: Andrés Limón Cruz <alc@datalabmx.com>
Date:   Thu Jan 9 11:33:11 2025 -0600

    Se agrega el archivo README
```

Extras

-¿Que pasa si eliminé por equivocación algún archivo de mi repositorio?

Afortunadamente git al ayudarnos a rastrear los cambios dentro de nuestro repositorio nota la eliminación de archivos, y podemos recuperarlos fácilmente con:

- git restore “Nombre del archivo”
- git restore . (Esto recupera todos los archivos borrados desde el ultimo cambio registrado en el repositorio)

-Tengo archivos que no me gustaría incluir para reportar mis cambios, ¿Cómo hago que git los ignore?

Para esto ocupamos un tipo de archivo especial para que git sepa que archivos no debe incluir, este tipo de archivo debe tener un nombre especial “.gitignore“ y en el agregamos el nombre de los archivos que no incluiremos o la terminación de archivos que ignoraremos

```
1  *.py
2  credenciales.txt
3
```

-Hice cambios en un archivo, pero quiero revertirlo a su estado original, ¿Cómo lo hago?

Si este es tu caso, git tiene una instrucción especial para revertir los cambios en los archivos, esta instrucción es: “git checkout – archivo”

-Añadí un archivo al área de preparación (stagged), ¿Cómo lo regreso al área de modificados?

En este caso se usa: “git reset archivo.txt”

-¿Cómo deshago algún commit?

Si tenemos algún commit que no queremos mantener lo podemos realizar de 2 maneras:

Perdiendo todos los cambios que se realizaron después de ese commit:

“git reset –hard HEAD~n”, con n indicamos que commit borraremos

Borrando solo ese commit y manteniendo los posteriores:

“git reset –soft HEAD~n”

```
(base) citrux@DELL-G15-5510:~/Desktop/test$ git log -1
commit 2146bd66ef465e4e5fb5a4f1f1e7d967539ceebc (HEAD -> jorgenohagascambiosenmain, origin/jorgenohagascambiosenmain)
Author: Andrés Limón Cruz <alc@datalabmx.com>
Date:   Mon Feb 17 09:55:32 2025 -0600

    Listo
(base) citrux@DELL-G15-5510:~/Desktop/test$ git reset --hard HEAD~1
HEAD is now at 20b6f53 Cambios
(base) citrux@DELL-G15-5510:~/Desktop/test$ git log -1
commit 20b6f538f9af89cf36dc739a6e69c435ff66fa7a (HEAD -> jorgenohagascambiosenmain)
Merge: 900987b 5fff0e9
Author: Andrés Limón Cruz <alc@datalabmx.com>
Date:   Mon Feb 17 09:35:31 2025 -0600

    Cambios
```

Último extra: Ramas (Branchs)

Una buena practica al momento de trabajar con git es el uso de ramas, podemos pensar en un repositorio git como un árbol, donde el tronco representa la rama maestra pues es la principal, ya que contiene la versión que sale a producción.

Si realizamos cambios de los cuales no estamos seguros sobre el efecto que pueden tener en la versión de producción lo mejor es usar una rama donde guardaremos nuestros cambios y en su momento uniremos de nuevo con la versión de producción.

Podemos verificar las ramas que existen en nuestro repositorio con: “git branch”

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git branch
* master
```

Podemos crear una nueva branch y verificar en que rama estamos

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git checkout -b nuevarama
Switched to a new branch 'nuevarama'
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git branch
  master
* nuevarama
```

Podemos volver a la rama master con: “git switch nombre_de_la_rama.º con “git switch nombre_de_la_rama”

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git switch master
Switched to branch 'master'
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git branch
* master
  nuevarama
```

Una vez que hemos hecho nuestros cambios en nuestra nueva rama, podemos integrar estos cambios sobre la rama master (ES NECESARIO CAMBIAR A LA RAMA SOBRE LA QUE SE UNIRÁN LAS RAMAS)

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git checkout master
Switched to branch 'master'
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git merge nuevarama
Updating 05af29a..b40aa38
Fast-forward
 .gitignore | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
```

Si ya no necesitamos la rama que hemos creado podemos eliminarla con “git branch -d nombre_de_la_rama”

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git branch
* master
  nuevarama
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git branch -d nuevarama
Deleted branch nuevarama (was b40aa38).
```

¿Pero como comparto mi proyecto?

Una opción mas fácil es iniciar el repositorio en algún gestor de repositorios en la nube como **GitHub** o **GitLab**, pero nuestro repositorio de ejemplo fue inicializado de manera local, por lo que para subir nuestros cambios a uno de estos gestores, necesitamos iniciar un repositorio vacío sobre ellos. Usamos la instrucción "git remote add origin url.git"Y luego podremos subir nuestros archivos con "git push origin master"

```
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git remote add origin https://github.com/Citruxs/Prueba.git
(base) citrux@DELL-G15-5510:~/Desktop/prueba$ git push origin master
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 541 bytes | 541.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Citruxs/Prueba.git
 * [new branch]      master -> master
```

Uso de GitHub

SI ya tenemos algun repositorio en GitHub, podemos clonarlo con “git clone url.git”

```
(base) citrux@DELL-G15-5510:~/Desktop$ git clone https://github.com/Citruxs/Prueba.git
Cloning into 'Prueba'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

Y verificar cambios en el repositorio en la nube con “git pull origin master”

```
(base) citrux@DELL-G15-5510:~/Desktop/Prueba$ git pull origin master
From https://github.com/Citruxs/Prueba
 * branch            master     -> FETCH_HEAD
Already up to date.
```

-¿Cómo descargo alguna rama disponible en la nube?
Si tenemos mas de una rama en nuestro repositorio en la nube, podemos cambiarnos a ella, pero lo primero es revisar que ramas tenemos disponibles con: git branch -a

```
(base) citrux@DELL-G15-5510:~/Desktop/test$ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/jorgenohagascambiosenmain
  remotes/origin/main
```

Una vez que ya hemos visto que ramas tenemos disponibles en nuestro repositorio, podemos cambiarnos a la rama que deseemos con git checkout nombre_de_la_rama

```
(base) citrux@DELL-G15-5510:~/Desktop/test$ git checkout jorgenohagascambiosenmain
branch 'jorgenohagascambiosenmain' set up to track 'origin/jorgenohagascambiosenmain'.
Switched to a new branch 'jorgenohagascambiosenmain'
(base) citrux@DELL-G15-5510:~/Desktop/test$ git branch
* jorgenohagascambiosenmain
  main
```

Anexo: Claves SSH

Si tenemos un repositorio privado o si queremos subir nuestros cambios a un repositorio en la nube, necesitamos configurar nuestra clave SSH.

Una clave SSH permite que los repositorios en la nube puedan verificar tu identidad sin necesidad de ingresar tu contraseña. Primero tenemos que configurar nuestra clave SSH con: "ssh-keygen -t rsa", este comando creará nuestras claves SSH pública y privadas en el directorio " /.ssh"

```
(base) citrux@DELL-G15-5510:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/citrux/.ssh/id_rsa):
/home/citrux/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/citrux/.ssh/id_rsa
Your public key has been saved in /home/citrux/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:qb5+L1ILSbQrcynw+tI8bbm+e7j0IzB/IV9ESjbLMWI citrux@DELL-G15-5510
The key's randomart image is:
+---[RSA 3072]----+
|                 |
|      E * .      |
|     o * B       |
|   .   o +..     |
|   o . +S.       |
|    B B.o .      |
|   + XoB +       |
|  o +oO.O        |
|   o.=X%.+.      |
+----[SHA256]-----+
```

Una vez generada la clave SSH, podmeos dirigirnos a la carpeta en la que ha sido creada

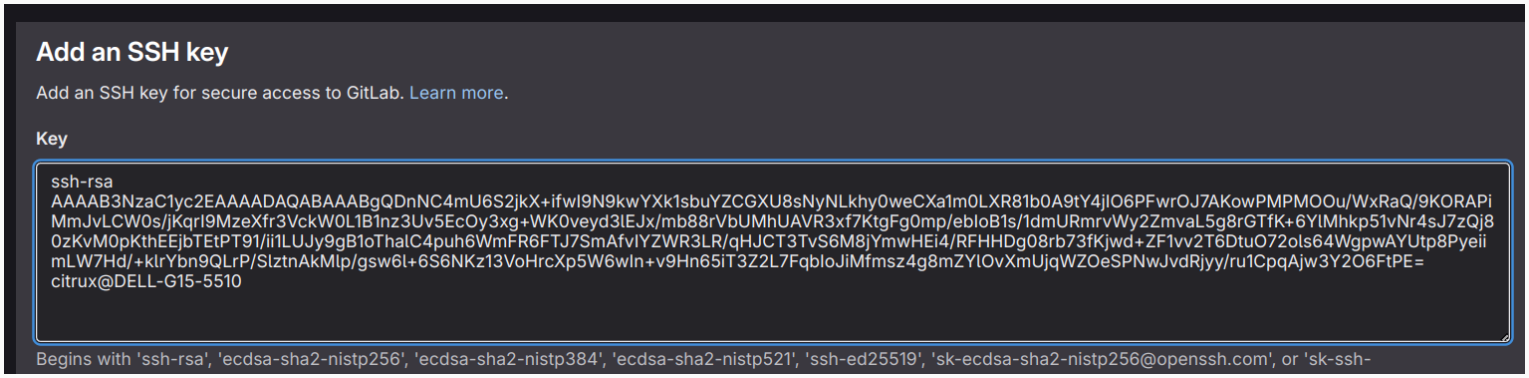
```
(base) citrux@DELL-G15-5510:~/Desktop$ cd ~/.ssh
(base) citrux@DELL-G15-5510:~/.ssh$ ls
id_rsa  id_rsa.pub  known_hosts
```

Y vemos el contenido de la llave publica id_rsa.pub

```
(base) citrux@DELL-G15-5510:~/.ssh$ cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDnNC4mU6S2jkX+ifwI9N9kwYXk1sbuYZCGXU8sNyNLkhy0we
MOOu/WxRaQ/9KORAPiMmJvLCW0s/jKqrI9MzeXfr3VckW0L1B1nz3Uv5EcOy3xg+WK0veyd3lEJx/mb88rVbUM
2ZmvaL5g8rGTfK+6YlMhkp51vNr4sJ7zQj80zKvM0pKthEEjbTEtPT91/ii1LUJy9gB1oThalC4puh6WmFR6FT
/RFHHDg08rb73fKjwd+ZF1vv2T6DtuO72ols64WgpwAYUtp8PyeiimLW7Hd/+klrYbn9QLrP/SlztnAkMlp/gs
2L7FqbIoJiMfmsz4g8mZYlOvXmUjqWZOeSPNwJvdRjyy/ru1CpqAjw3Y2O6FtPE= citrux@DELL-G15-5510
```

Copiamos el resultado y nos dirigimos a nuestro gestor de repositorios en la nube como **GitHub** o **GitLab**

Agregamos la llave SSH en la sección de configuración del administrador de repositorios



Add new SSH Key

Title

Fedora_Citrux

Key type

Authentication Key

Key

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQDnNC4mU6S2jkX+ifwI9N9kwYXk1sbuYZCGXU8sNyNLkhy0weCXa1m0LXR81b0A9tY4jIO6PFwrOJ7
AKowPMPMOOu/WxRaQ/9KORAPiMmJvLCW0s/jKqrI9MzeXfr3VckW0L1B1nz3Uv5EcOy3xg+WK0veyd3lEJx/mb88rVbUMhUAVR3xf7KtgFg0
mp/ebIoB1s/1dmURmrvWy2ZmvaL5g8rGTfK+6YlMhkp51vNr4sJ7zQj80zKvM0pKthEEjbTEtPT91/ii1LUJy9gB1oThalC4puh6WmFR6FTJ7SmAf
vIYZWR3LR/qHJCT3TvS6M8jYmwHEi4/RFHHDg08rb73fKjwd+ZF1vv2T6DtuO72ols64WgpwAYUtp8PyeiimLW7Hd/+klrYbn9QLrP/SlztnAkMlp/
gsw6l+6S6NKz13VoHrcXp5W6wIn+v9Hn65iT3Z2L7FqbIoJiMfmsz4g8mZYlOvXmUjqWZOeSPNwJvdRjyy/ru1CpqAjw3Y2O6FtPE=
citrux@DELL-G15-5510

Add SSH key